

Matematično-fizikalni praktikum: FFT in avtokorelacija

Žan Ambrožič (28231062)

17. november 2025

1 Naloga

Na spletni strani MF praktikuma najdeš posnetke oglašanja velike uharice, naše največje sove. Posneti sta dve sovi z minimalnim ozadjem (`bubomono` in `bubo2mono`) in nekaj mešanih signalov, ki zakrivajo njuno oglašanje (`mix`, `mix1`, `mix2` in `mix22`). V signalih `mix2` in `mix22` je oglašanje sove komaj še zaznavno. Izračunaj avtokorelacijsko funkcijo vseh signalov in poskusi ugotoviti, za katero sovo gre pri teh najbolj zašumljenih signalih!

Poglejte si rutine `four1` iz Numerical Recipes ali knjižnice `fftw3`, ki je še dosti hitrejša. V okolju Python so te rutine vključene v `'fft'` paket. (Pri tako velikih vzorcih je skorajda nujno uporabiti FFT namesto počasne navadne DFT.)

2 Uvod

Diskretno Fourierovo transformacijo smo definirali kot

$$H_k = \sum_{n=0}^{N-1} h_n \exp\left(2\pi i \int kn/N\right), \quad k = -\frac{N}{2}, \dots, \frac{N}{2},$$

oziroma

$$H_k = \sum_{n=0}^{N-1} W_N^{nk} h_n, \quad W_N = \exp\left(2\pi i \int /N\right).$$

Ta postopek ima očitno časovno zahtevnost N^2 . Račun pa je mogoče izvesti tudi z bistveno manj operacijami. Osnovni premislek je razcep

$$H_k = H_k^{\text{sod}} + W_N^k H_k^{\text{lih}},$$

kjer smo transformiranko H izrazili s transformacijama njenih sodih in lihih členov, pri čemer je vsota vsake od transformacij zdaj dolžine $N/2$. Gornjo relacijo lahko uporabljamo rekurzivno: če je N enak potenci števila 2, lahko rekurzijo razdrobimo do nizov, ki imajo samo še en člen. Zanj je transformacija identiteta. Za obrat pri eni vrednosti frekvence (pri danem m) je potrebno na vsakem koraku rekurzije le eno množenje s potenco W , korakov pa je $\log_2 N$. Skupna časovna zahtevnost je torej le še $N \log_2 N$.

Da ne iščemo pripadnikov niza po vsej tabeli, si podatke preuredimo. Lahko je pokazati, da je v prvotni tabeli treba med seboj zamenjati podatke, katerih vrstna števila v binarnem zapisu so obrnjena: v novem redu jemljemo člene kar po vrsti. Tudi potenc W ne izražamo vedno znova s sinusi in kosinusi, pač pa jih računamo z rekurzijo. Tak ali podoben postopek je osnova vseh algoritmov hitre Fourierove transformacije (FFT).

Z neko transformacijo iz družine FFT bomo izračunali korelacijsko funkcijo dveh signalov. Korelacija periodičnih funkcij $g(t)$ in $h(t)$ s periodo T je definirana kot:

$$\phi_{gh}(\tau) = \frac{1}{T} \int_0^T g(t + \tau) h(t) dt,$$

oziroma diskretno

$$\phi_{gh}(n) = \frac{1}{N} \sum_{k=0}^{N-1} g_{k+n} h_k.$$

Računamo torej skalarni produkt funkcij, ki sta časovno premaknjeni za τ oziroma n . Če je za določeno vrednost premika ta funkcija višja kot v okolici, potem to pomeni, da sta si funkciji podobni, le da ju je treba premakniti, da se to vidi.

V primeru, da sta funkciji (signala), ki ju primerjamo, enaki, računamo njuno *avtokorelacijsko funkcijo*: ta je mera za to, ali signal ostaja s pretakanjem časa sam sebi podoben. Če je signal slabo koreliran (sam s sabo), korelacija $\phi_{hh}(n)$ relaksira h kvadratu povprečnega signala $\langle h \rangle^2$, kjer je

$$\langle h \rangle = \frac{1}{N} \sum_{k=0}^{N-1} h_k .$$

Iz lokalnih maksimov v avtokorelacijski funkciji sklepamo na periodičnosti, bodisi popolne ali približne. Pri periodičnih signalih je tudi avtokorelacijska funkcija striktno periodična, za stohastične procese pa je značilna eksponentna avtokorelacijska funkcija. Še bolj nas zanima, kako *hitro* se korelacija izgublja: računamo rajši reskalirano obliko avtokorelacije

$$\tilde{\phi}_{hh}(n) = \frac{\phi_{hh}(n) - \langle h \rangle^2}{\phi_{hh}(0) - \langle h \rangle^2} ,$$

kjer je imenovalac nekakšno merilo za varianco signala,

$$\sigma^2 = \phi_{hh}(0) - \langle h \rangle^2 = \frac{1}{N} \sum_{k=0}^{N-1} (h_k - \langle h \rangle)^2 .$$

Pri zgornjih enačbah moramo še “peš” poskrbeti za periodično zaključenost signala pri $n = N$, torej da je perioda enaka velikosti vzorca. Če tega ne moremo narediti, je bolj pravilna definicija avtokorelacije

$$\phi_{hh}(n) = \frac{1}{N-n} \sum_{k=0}^{N-n-1} h_{k+n} h_k .$$

Praktičen račun po zgornji formuli lahko postane za velike vzorce prezauden. Avtokorelacijo rajši računamo s FFT (DFT) $\mathcal{F}$, saj je korelacija obratna Fourierova transformacija $\mathcal{F}^{-1}$ produkta Fourierovih transformacij $\mathcal{F}$, torej z $G = \mathcal{F}g$ in $H = \mathcal{F}h$ dobimo

$$\phi_{gh}(n) = \frac{1}{N-n} \mathcal{F}^{-1} [G \cdot (H)^*]$$

oziroma

$$\phi_{hh}(n) = \frac{1}{N-n} \mathcal{F}^{-1} [|H|^2] .$$

Za račun s FFT signale dolžine N najprej prepišemo v dvakrat daljše, periodično zaključene podatkovne nize, $\tilde{h}_n = h_n$, $\tilde{h}_{n+N} = 0$ za $n = 0, \dots, N-1$ in $\tilde{h}_{n+2N} = \tilde{h}_n$. Tedaj se avtokorelacija zapiše v obliki

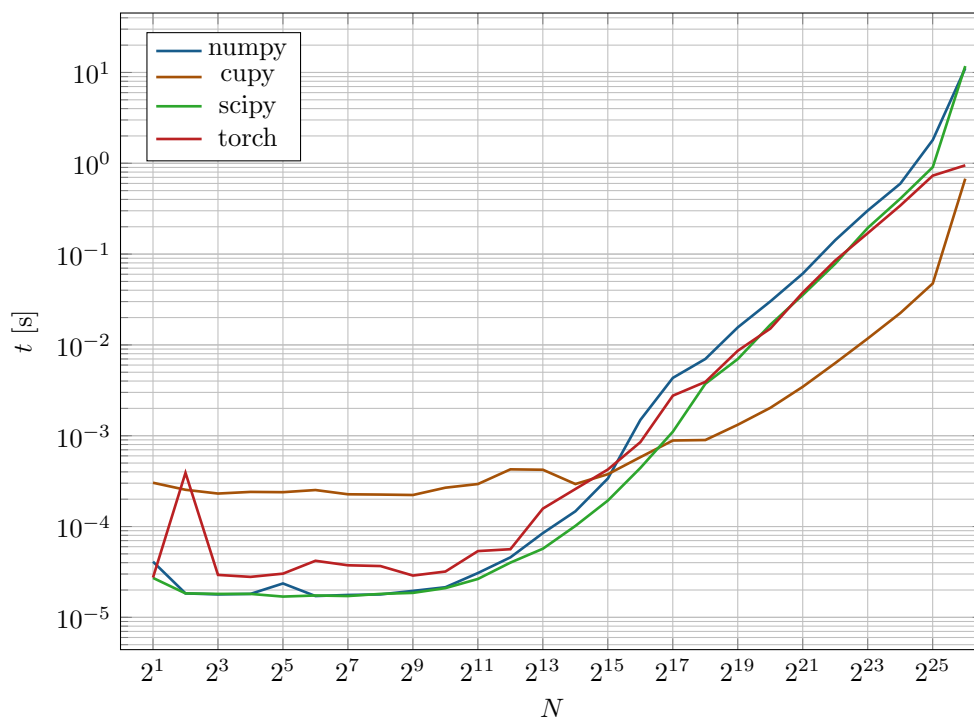
$$\phi_{hh}(n) = \frac{1}{N-n} \sum_{k=0}^{2N-1} \tilde{h}_{k+n} \tilde{h}_k ,$$

kar lahko izračunamo s FFT.

3 Testi in rezultati

3.1 Časovna zahtevnost FFT

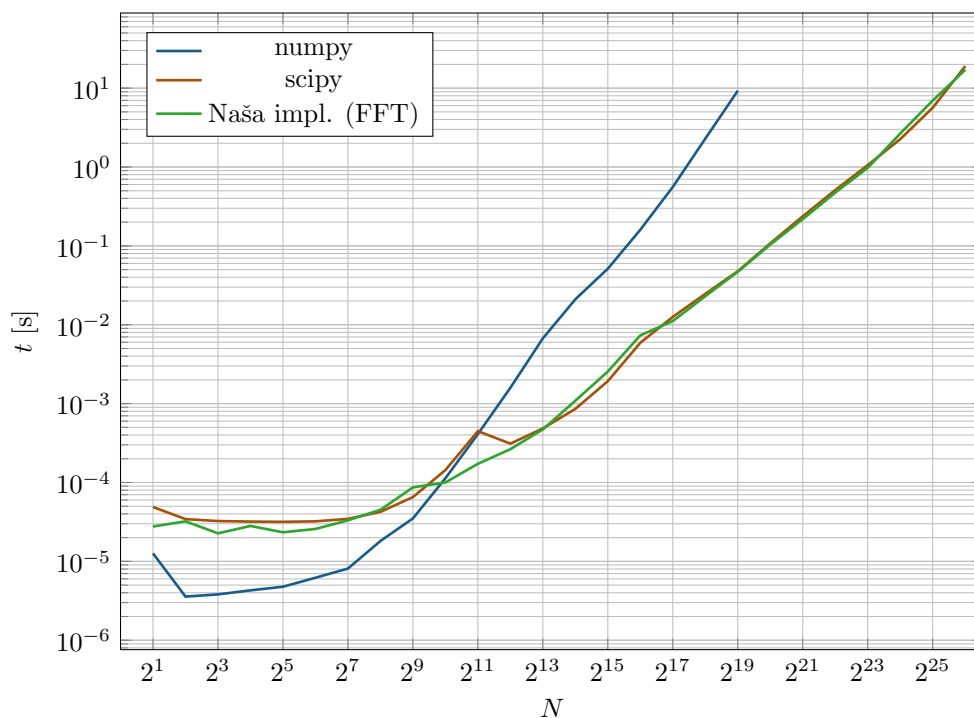
Najprej bomo preizkusili različne implementacije FFT po hitrosti glede na dolžino signala. Na CPU (AMD Ryzen 7 5800x) bomo preizkusili `numpy.fft.fft` in `scipy.fft.fft`, na GPU (Nvidia RTX 3060, 12 GB) pa `cupy.fft.fft` ter `torch.fft.fft`. Pri tem bomo (za `cupy` in `torch`) vnaprej pretvorili podatke iz `numpy.ndarray` v primerna zapisa za knjižnici, po vsaki pretvorbi pa bomo tudi počistili spomin GPU (s primernima ukazoma iz knjižnic, ki uporabljata GPU). Pred začetkom merjenja časa bomo že enkrat pognali vse metode na kratkem signalu, ker je sicer prvi zagon `cupy` vedno zamuden in dobimo lažno špico (verjetno posledica nalaganja/prevajanja strojne kode za GPU). Čas merimo le za operacijo FFT, ne za prej opisane postopke. Za testni signal vzamemo kar enega od zapisov `bubomono.txt` in ga nekajkrat podvojimo, da dobimo vzorce tudi za večje N . Za GPU operacije moramo za pošteno primerjavo pred končanjem merjenja časa zahtevati sinhronizacijo, saj nekatere metode (tudi FFT) tečejo asinhrono, kar nam da lažen čas. Ker FFT na GPU najprej faktorizira vzorce, lahko dobimo špice tudi do 100-kratne razlike v časih izvedbe (pri kakšnih manj posrečenih dolžinah vzorcev), zato predpostavimo optimalen scenarij in naredimo vzorce dolžin potenc števila 2 (do 26, takrat dosežemo okoli 8 GB, zato bi naslednja podvojitev prekoračila kapaciteto GPU).



Graf 1: Časovna zahtevnost implementacij FFT štirih knjižnic na različno dolgih signalih.

3.2 Časovna zahtevnost avtokorelacije

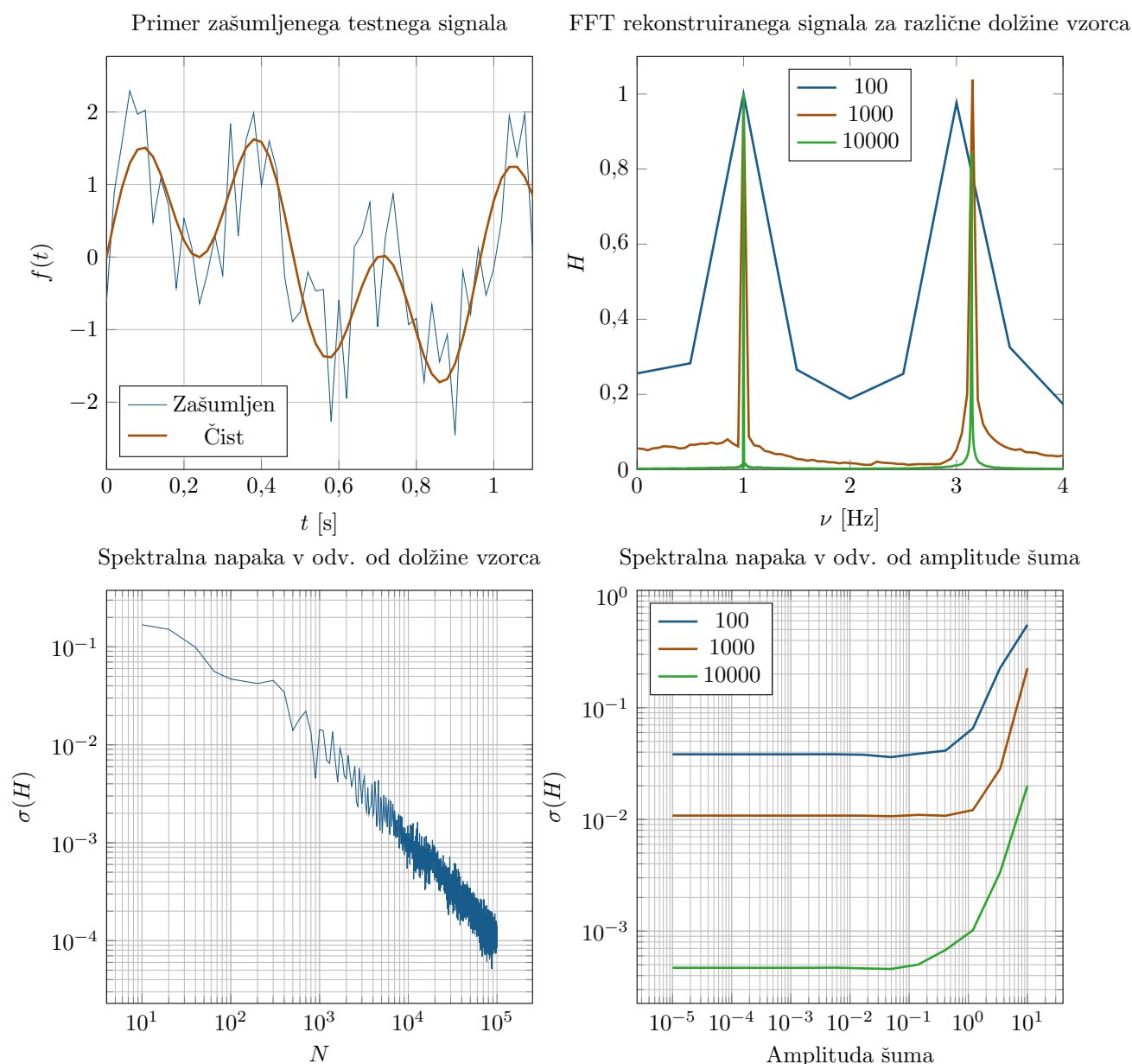
Za avtokorelacijo imamo že pripravljene nekatere vgrajene metode, ki se za razliko od FFT precej razlikujejo po postopku: medtem ko `numpy.correlate` računa s signalom, `scipy.signal.correlate` uporablja FFT (oz. v avtomatskem načinu oceni, katera metoda je hitrejša). Naredimo še našo enostavno implementacijo, ki naredi FFT, ga pomnoži s svojim konjugiranim signalom, ter naredi obratno FFT (uporablja `scipy.fft.fft`, ker se je izkazal za malo hitrejšega od `numpy` inačice). Trenutna verzija `scipy` očitno tako za FFT kot za svojo implementacijo korelacije uporablja eno nit procesorja in še malo pomoči GPU, medtem ko `numpy.correlate` teče na večih nitih vzporedno. Z novjšimi verzijami `numpy` (ki tečejo vzporedno) so na tem računalniku že bile težave (3. naloga – diagonalizacija veliki matrik) in pojavile so se tudi tu, ker omejitev na 8 niti ni zadoščala in je računalnik začel stavkati kmalu po zagonu izračunov, je bila varnejša omejitev 4/16 niti, kar je precej upočasnilo izračune (to je treba vzeti v zakup pri spodnjem grafu, saj bi časi lahko bili do štirikrat krajši). Za testni signal uporabimo interval x dolžine 100, ki ga razdelimo na 2^{26} enakih delov, nato pa za signal izračunamo vrednosti sinusa x ter πx (ni večkratnik prvega) z amplitudama 1, dodamo pa še naključen šum enake amplitude.



Graf 2: Časovna zahtevnost implementacij avtokorelacije na različno dolgih signalih.

3.3 Natančnost avtokorelacije

V tem delu nas bo zanimala natančnost rekonstruiranega signala iz zašumljenega vzorca. Rekonstrukcijo bomo delali na signalu iz prejšnjega dela različnih dolžin in z različnimi stopnjami šuma. Vzeli bomo korak 0,02 ter sinusa s frekvencama 1 in π . Naredimo FFT vzorca, ne glede na dolžino vzorca amplitude normiramo tako, da je vrh pri $\nu = 1$ vedno 1. Te vrednosti nato primerjamo z vrednostmi FFT nezašumljenega signala. Za izračun avtokorelacij uporabljamo `scipy.signal.correlate`, ki daje enake rezultate ko `numpy.correlate`, vendar je znatno hitrejši pri daljših signalih. Ročno moramo dodati še normalizacijo $1/(N - n)$.



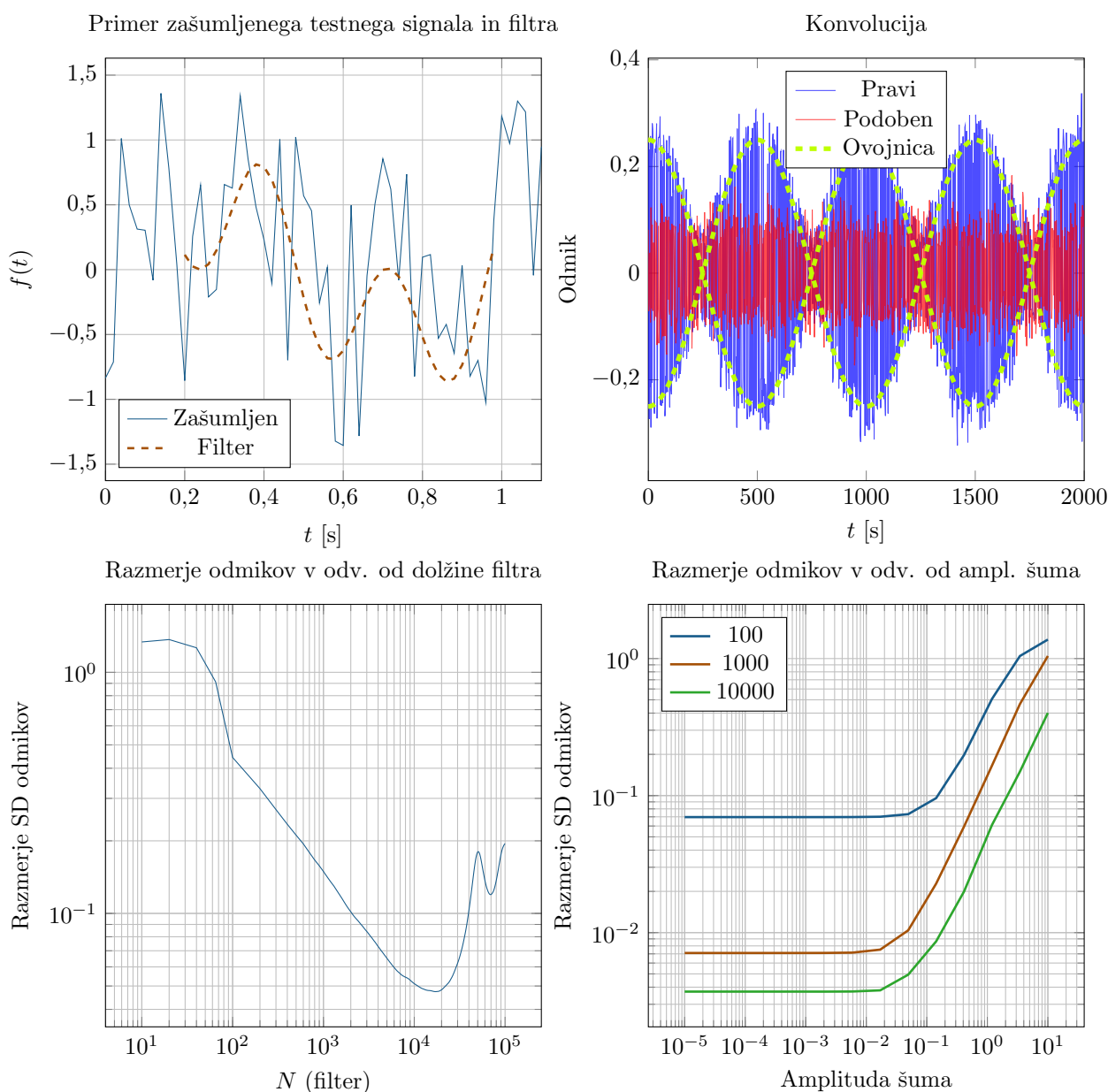
Graf 3: Primer zašumljenega vhodnega signala in spektralna natančnost rekonstrukcije z avtokorelacijo v odvisnosti od dolžine signala in amplitude šuma.

Pričakovano spektralna napaka pada z daljšanjem idealno periodičnega vzorca, šum pa je do neke amplitude zanemarljiv, nato pa počasi z naraščanjem začne kvariti rezultate.

3.4 Korelacija dveh signalov

Dva signala lahko primerjamo tudi direktno s konvolucijo, pri tem lahko enemu (običajno krajšemu) signalu rečemo filter, s katerim gremo čez drugi signal (da pridemo čez celotno območje, običajno preiskovanemu signalu dodamo za dolžino filtra ničel). Za preizkus metode vzamemo enak signal kot prej, vendar nezašumljeni komponenti pomnožimo s kosinusom, ki ima periodo 1000 s (polovico dolžine vzorca) ter normaliziramo največji možni odmik na 1 (enostavno delimo z 2 zaradi dveh sinusov). Za pravi signal vzamemo le sinusni komponenti (s polovično amplitudo, da je enako normaliziran), za "podoben" signal pa vzamemo sinus s frekvenco 1,5 Hz, kar je med obema komponentama pravega signala. Za pravi signal moramo dobiti nazaj kosinus s periodo 1000 s in amplitudo 0,25 (z malo šuma, ki bi se čez nekaj točk moral izpovprečiti), za "podoben" signal pa pričakujemo neko manjšo amplitudo, ki je odvisna od dolžine filtra (glede na periode komponent) – več period, kot vsebuje filter, bolj natančen je v primerjavi z nepravim. Seveda moramo zaradi naše kosinusne ovojnice paziti, da je filter krajši od periode kosinusa, saj bi se sicer amplituda

konvolucije pravega signala zmanjšala, medtem ko se za nepravi signal v večini primerov ne bi dosti spremenila, zato bi za daljše filtre dobili artefakt poslabšanja natančnosti napovedi. Za signal vzamemo časovno domeno $t \in [0, 2000 \text{ s}]$, ter število točk $N_0 = 10^5$. Za natančnost bomo vzeli cenilko razmerja standardnih deviacij odmikov konvolucijskih točk za pravi in "podoben" filter (kakršna sta opisana).

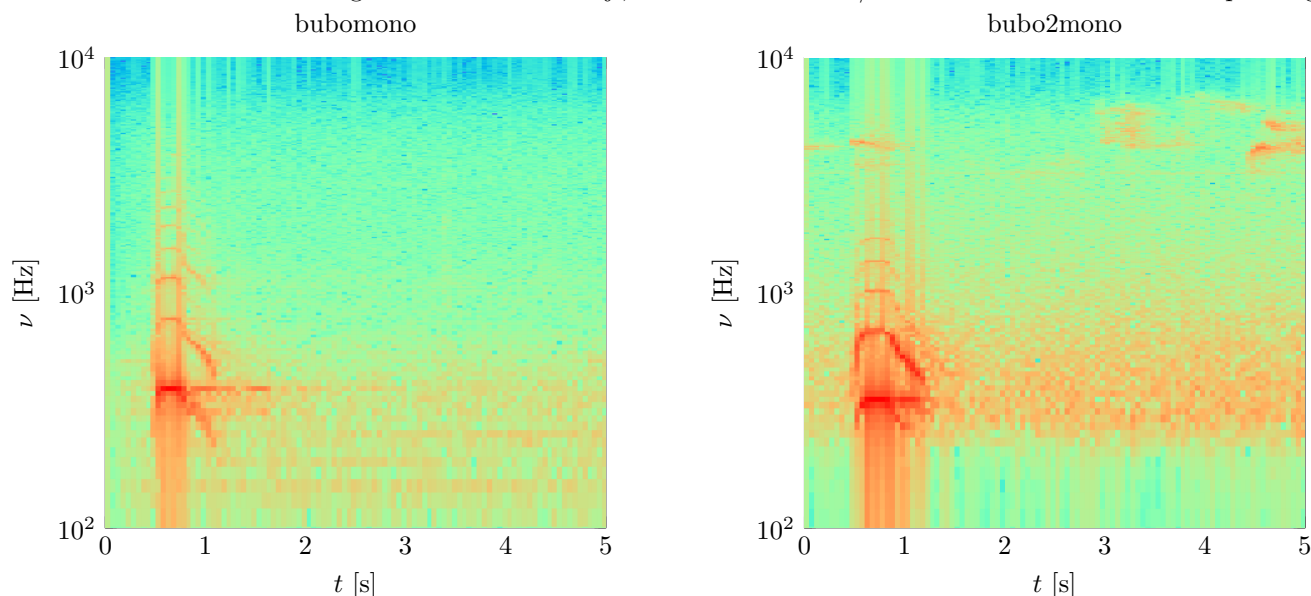


Graf 4: Primer zašumljenega testnega signala in pravega filtra za signal, konvolucija opisanih filtrov dolžine $N = 100$ s kosinusno ovojnico četrtnine amplitude testnega signala. Druga dva grafa prikazujeta rezultate testov natančnosti filtra v odvisnosti od dolžine le tega (razmerje SD amplitud za "podoben" in pravi signal), z enako cenilko primerjamo tri različno dolge filtre na različno močno zašumljenih signalih (amplituda šuma 1 je enaka največji možni amplitudi testnega signala).

Kot pričakovano filter postaja natančnejši za dolžine do petine dolžine celotnega testnega signala, se pa šum bolj pozna kot pri metodi z avtokorelacijo (tam smo sicer uporabili šum, ki je imel relativno dvakrat večjo amplitudo glede na amplitudo signala, signal pa ni imel še kosinusne ovojnice, kar bi bilo potrebno upoštevati pri testih).

3.5 Sove

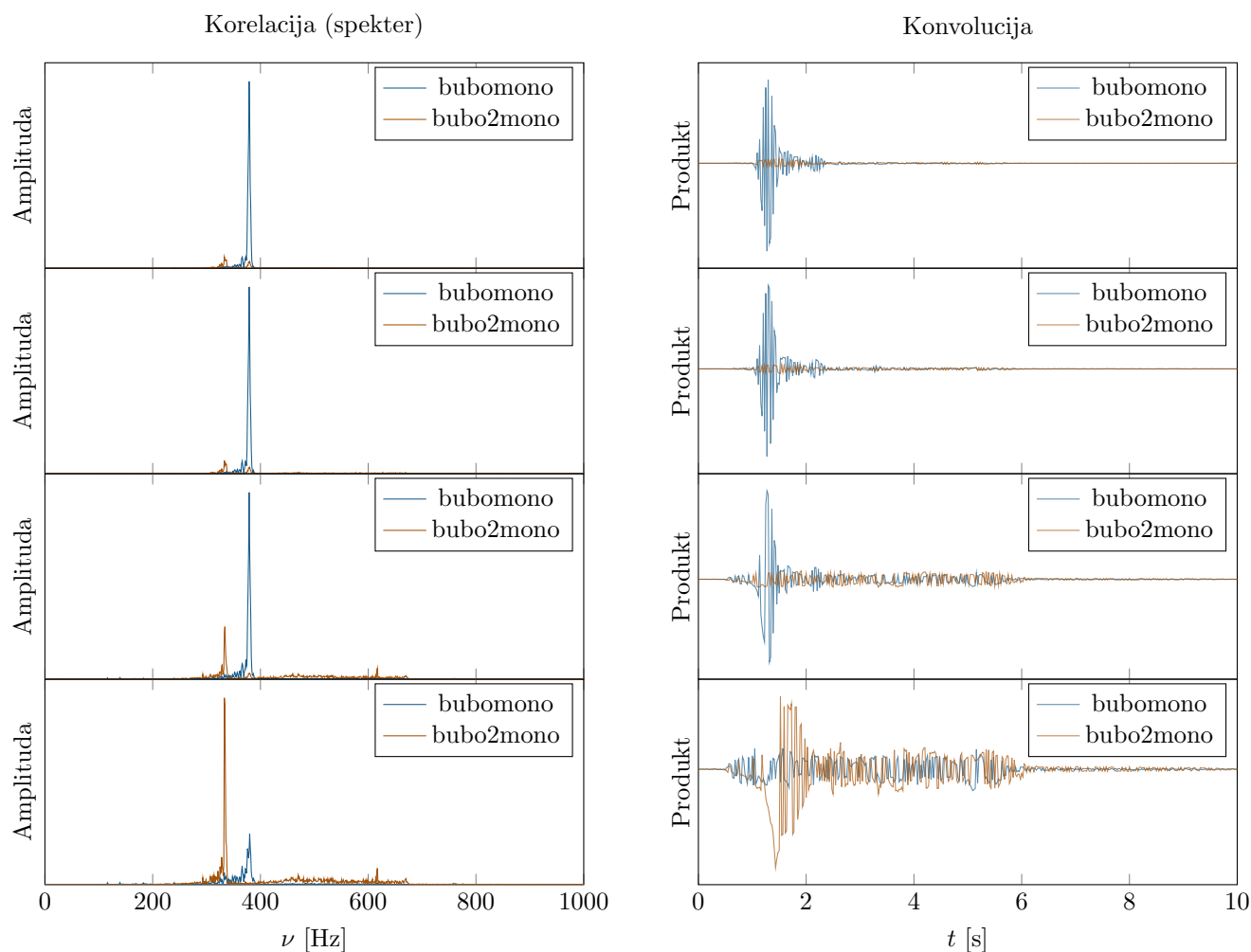
Za praktičen preizkus metod bomo analizirali zašumljene posnetke sov, ki jih bomo korelirali s čistima posnetkoma dveh različnih sov ter poskušali določiti, katero je slišati na posameznem posnetku. Za začetek si oglejmo spektrograma čistih posnetkov, posnetki so zapisani s hitrostjo 44,1 kHz in dolgi po 5 s. Pri analizi Bachove Partite smo uporabljali časovno resolucijo 40/s in dopolnili trikratno dolžino signala z ničlami, da smo izboljšali frekvenčno resolucijo, da smo lahko opazili vse zaigrane tone (tako časovno kot za razpoznavno višine tonov). Ker tokrat nimamo tako strogih zahtev za resoluciji, lahko vzamemo 20/s in naredimo FFT brez zero-padding-a.



Graf 5: Spektrograma čistih posnetkov dveh sov s časovno resolucijo 0,2 s.

Na spektrogramih opazimo (kakor lahko tudi slišimo na posnetkih), da se sovi oglašita na začetku posnetkov ter da je druga sova nekoliko nižja kot prva (po višini tona oglašanja). Lepo vidimo tudi višje resonančne frekvence, ki nakazujejo, da sovi proizvajata glasove bolj podobno pol-zaprti cevi kot pa na obeh strani odprti/zaprti. Če analiza celotnih zašumljenih posnetkov s celotnima čistima posnetkoma ne bo dala dovolj dobrih rezultatov, jih lahko izboljšamo z uporabo krajših signalov – izrežemo le dele, kjer se sove sliši in jih analiziramo.

Posnetke bomo analizirali po obeh opisanih metodah – s korelacijo in s konvolucijo. Pri korelaciji si bomo ogledali spekter izhodnega signala, kjer pričakujemo spekter, podoben čistima posnetkoma obeh sov (seveda je treba upoštevati celotne dolžine, zato se morda nekatere višje frekvence izgubijo, kot omenjeno bomo v primeru potrebe po boljši analizi vzeli krajše odseke). Za boljšo primerjavo najprej normaliziramo čista posnetka na enako RMS amplitudo, da ne bi dobili boljšega odziva pri eni le zato, ker je amplituda posnetka višja. Ker gre za zvočne posnetke, nam ni potrebno paziti na morebiten odmik (offset) celotnega posnetka. Za izboljšanje zaznave bi lahko glede na spektrograma porezali frekvence nekje pod 200 Hz ter nad 2000 Hz, saj tam ni zaznati sove, lahko pa je del šuma v teh območjih. Posnetki so vsi enako dolgi, zato nam ni treba tako zelo paziti na $1/(N - n)$ normalizacijo.

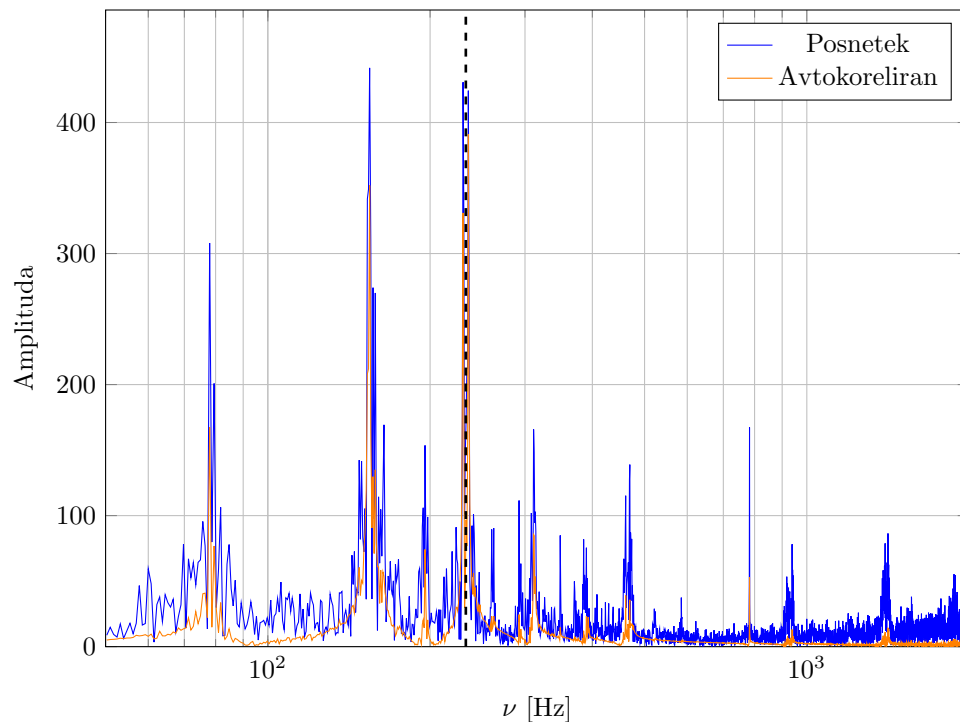


Graf 6: Analiza štirih posnetkov sov s korelacijo in konvolucijo. Pravilno razpoznamo (večji signal), da so prvi trije primeri prve sove, zadnji pa je primer druge. Grafi so za posnetke po vrsti od zgoraj navzdol: `mix`, `mix1`, `mix2` in `mix22`.

Kot napovedano pri korelaciji dobimo zaradi uporabe celotnih posnetkov le najnižji frekvenci, se pa iz razmerja signalov dobro da razpoznati sove in dodatne augmentacijske metode in obrezovanje posnetkov ni potrebno. Tudi konvolucija nam da dobre rezultate brez obrezovanja, je pa kot napovedano dolžina izhodnega signala za dve dolžini vhodnih signalov, saj gremo s filtrom čez za celotno prekrivanje čez signal, efektivno podaljšan z ničlami.

3.6 Čiščenje aplavza s posnetka

Včasih se lahko na koncertih aplavz zgodi med skladbo (običajno po solih v zabavni glasbi), ki bi ga želeli očistiti s posnetka. Za vzorec vzamemo posnetek saksofonista Erica Marienthala, ki v živo igra *I Believe In You* [1]. Proti koncu skladbe zadrži srednji G na E $\flat$ saksofonu (okoli 233 Hz), na posnetku je slišati še stalen zven činele (rahlo vzbujanje s "tušom") ter klaviature na nizkem tonu. Z avtokorelacijo štiri-sekundnega posnetka (6:32–6:36) pri vzorčenju z 22 050 Hz uspešno odstranimo aplavz, se nam pa zelo ojačijo spodnji ton klaviatur, činela in saksofon, ki imajo bolj kot ne stalne frekvence tonov (saksofonist dela rahel vibrato, vendar to ne zmoti postopka). Klaviatur se v tem primeru znebimo enostavno tako, da porežemo nižje frekvence (recimo pod 220 Hz), s tem pa dobimo dobro izoliran saksofon in zven činele. Če se želimo znebiti še slednje, porežemo frekvence nekje nad 780 Hz, vendar pa to vpliva tudi na višje harmonike saksofona, ki se s tem izgubijo in ton nima več barve saksofona.



Graf 7: FFT originalnega in avtokoreliranega (brez $1/(N - n)$ normalizacije) štirisekundnega odseka s koncerta, kjer smo z avtokorelacijo porezali aplavz in izolirali inštrumente, ki držijo stalen ton. S črtkano črto je označena osnovna frekvenca tona, ki ga igra saksofonist.

Zanimivo je, da je izhodni signal slišati praktično enako ne glede na to, ali uporabimo normalizacijo ali ne (seveda je signal treba primerno ojačiti, da je na koncu enako glasen), se pa pozna razlika na koncu – amplituda normaliziranega hitro naraste, med tem ko se pri nenormaliziranem počasi zaduši (*fade-out*), nič od tega pa ni enako kot na originalnem posnetku – oba efekta sta očitno artefakta našega postopka, se pa tudi ne poznata veliko na spektrogramu.

4 Zaključek

V tej nalogi smo raziskali osnovne lastnosti avtokorelacije in korelacije signalov. Preizkusili smo različne knjižnice, ki omogočajo računanje le teh ter preverili, kako dobro se da enostavne signale rekonstruirati iz zašumljenih glede na razmerje med čistim signalom in šumom ter glede na dolžine vzorcev. Za realni preizkus smo klasificirali sovo po oglašanju na zašumljenih posnetkih na podlagi čistih, ter na odseku koncertnega posnetka uspešno odstranili aplavz, ki je neperiodičen signal in je zato podoben šumu, za razliko od periodičnih signalov, ki jih oddajajo glasbila.

Literatura

- [1] YouTube. *Eric Marienthal: I Believe In You (live)*. Accessed: 16 November 2025. 2011. URL: <https://www.youtube.com/watch?v=-N1b-NKL7gw>.